TECHNICAL HANDOVER · SOLL-IST-ANALYSE

OneAlarm

A personal iOS app that propagates one wake time to an iPhone, an Eight Sleep Pod and a Whoop strap. This document is the full state of it: what it is meant to do, what it actually does, how the Eight Sleep write was solved, and every approach that was tried and failed.

Written 18 August 2026 For external engineers joining the work

Platform iOS 26, Swift 6, SwiftUI Backend none

START HERE

The three things we want help with

Everything below is context for these. If you read nothing else, read this section and the Soll-Ist table.

> **Question 1 was answered hours after this document was first written**, by reading two maintained clients at source rather than reasoning further. It is struck through rather than deleted, because how it was answered is the most useful thing here: the answer had been sitting unread in this project's own notes, and what closed it was going to look rather than thinking harder.

~~1. How does Eight Sleep represent a one time change?~~

Answered late on 18 August, and left in place because how it was answered still matters. The question was where `UPCOMING ALARM ONLY` lives, given that four raw dumps of the alarm object show no field carrying it.

It is not a field. A one-off is **its own alarm, created by omitting the `repeat` block**. Confirmed in the maintained Home Assistant integration, which has a function for exactly this:

```
async def set_one_off_alarm(self, time, enabled=True, vibration_...,
thermal_...):
    """Create a new alarm via the new alarms API."""
    url = APP_API_URL + f"v1/users/{self.user_id}/alarms"
    data = {"time": time, "enabled": enabled,
            "vibration": {...}, "thermal": {...}}
    await self.device.api_request("POST", url, data=data)
```

The same file's alarm reader carries the comment *"not just recommendedAlarm, which may skip one-off alarms"*, so one-offs are first-class entries in `alarms[]` rather than a flag on a recurring alarm. That is a **second source** for a line sitting unread in this project's own research file since day one, so it meets the two-source bar.

Still open, and only the hardware answers it: whether a one-shot clears itself after firing. If it does, the ownership key, the link and the delete described in notes 3 and 4 all become unnecessary.

2. Does Whoop have a create or delete endpoint for smart alarm schedules?

Updating a schedule is confirmed working. **No public source documents a create or a delete**, including the most thorough public capture of the app, whose author deliberately exercised the CRUD and recorded only an update. We ship a ladder of candidate requests. It has never been confirmed.

3. Is exact day-set matching the wrong identity model, and what replaces it?

The owner of the app put it precisely: *"if you set it up to match the actual settings in the apps then it usually works, because then it gets the right data. The problem is when something changes."* Every failure has that shape. Detail in footnote 8.

PRODUCT

What the app is

One wake time, set in one place, applied to every device that can wake a person. The three devices fire in sequence rather than together, because they do different jobs:

DEVICE	OFFSET	WHAT IT DOES	MECHANISM
Eight Sleep Pod	master minus 10 min	Thermal and vibration wake, starts before the noise	Private HTTP API, reverse engineered
Whoop strap	master minus 5 min	Haptic smart alarm inside a wake window	Private HTTP API, reverse engineered
iPhone	master	The alarm that must ring. No account, no network, no server	AlarmKit, Apple framework

The app is built around **routines**, not a single alarm. A routine is a set of weekdays plus a time: "Weekdays 06:15", "Weekend 09:30". Each routine drives one alarm on each device. On top of that sit two one-day operations: a **bend** (tomorrow only, at a different time) and a **skip** (no alarm tomorrow, routine untouched).

The product rule that governs everything: **OneAlarm is the single source of truth for alarm scheduling. Only comfort settings are edited in the device's own app.** Temperature, vibration strength and pattern belong to Eight Sleep's app and are read from the server and handed straight back on every write. OneAlarm authors exactly three fields on an Eight Sleep alarm it owns: `time`, `repeat.weekDays` and `enabled`.

ENGINEERING

How it is built

Language	Swift 6, strict concurrency. Adapters are actors.
UI	SwiftUI, iOS 26 minimum (AlarmKit requires it).
Persistence	`UserDefaults` for schedule and ownership maps. iOS Keychain for all credentials.
Network	`URLSession` only. No third party SDKs, no analytics, no backend of any kind.
Testing	XCTest against a `URLProtocol` stub server. Roughly 120 tests. No device tests.

Shape of the code

The decision-making is pure and the I/O is thin, deliberately, because the interesting bugs are all in the decisions:

- `RulesEngine` is a pure function. It takes a schedule, a calendar and an instant, and returns one `ResolvedTarget` per device plus a `RoutinePlan`: the whole week, projected onto that device's clock with its offset applied. It has no clock of its own, which is what makes midnight and daylight-saving edges testable.
- `RoutinePlan.match` pairs each routine with the alarm it owns on a service.
- `AlarmKitReconciler` is pure and decides which phone alarms to schedule and which to cancel. `AlarmManager.shared` is an Apple singleton with no seam, so the adapter is a shell that does what it is told.
- Three adapters conform to one `DeviceAdapter` protocol: `preview`, `write`, `verify`.

Two safety mechanisms worth knowing about

Ownership is recorded, not inferred. `RemoteAlarmLink` maps a routine ID to a remote alarm ID, and separately records which alarm IDs OneAlarm itself created. Only an ID on that second list may ever be deleted. An alarm the user made by hand, or one OneAlarm adopted because its days happened to match, is never deleted whatever state it is in. The most that happens to it is being switched off, which is reversible in one tap in the device's own app.

Every write is read back. A 200 is never treated as a moved alarm. Eight Sleep returns `nextTimestamp`, an absolute instant, so the write is verified against the

moment we actually meant. Whoop returns no such field, so its best possible state is permanently "saved" rather than "confirmed", and the app says so rather than implying more.

> **An operating constraint worth stating, because it shapes everything below.** Development happens in an environment with no Swift compiler and no network route to any of the three services. Both re-verified on 18 August rather than assumed. Every type error and every question about what a server accepts is therefore answered on the owner's phone, which makes each round trip expensive and makes the quality of the app's own reporting the main lever on progress.

THE HARD PART

Eight Sleep, and how the write was solved

This leg took roughly two weeks and produced every interesting failure in the project. It works now for routines. The account details below are redacted.

AUTH

Password grant against `auth-api.8slp.net`, one attempt only, never retried. Token and user ID cached in memory, credentials in the Keychain. Three hosts, and mixing them up is a real error class:

HOST	USED FOR
`auth-api.8slp.net`	Token issue only
`client-api.8slp.net`	Account, device list, bed name and side
`app-api.8slp.net`	Alarms, routines, temperature

THE ALARM ENDPOINTS, WHICH ARE VERSION-INCONSISTENT

```
GET    https://app-api.8slp.net/v2/users/{userId}/alarms
PUT    https://app-api.8slp.net/v1/users/{userId}/alarms/{alarmId}
```

```
POST   https://app-api.8slp.net/v1/users/{userId}/alarms
DELETE https://app-api.8slp.net/v1/users/{userId}/alarms/{alarmId}
```

Reads are v2 and writes are v1. That is not a transcription error, it is confirmed against the live account. The DELETE is a convention rather than a capture: no public source documents it, so a refusal is reported with its status rather than swallowed, and the alarm is switched off instead.

THE WRITE IS A READ-MODIFY-WRITE OF THE WHOLE OBJECT

It replaces rather than merges. Take the object from the list, strip five server-computed fields, replace the fields you own, send everything else back **including fields with no known meaning**. The five to strip:

```
nextTimestamp  startTimestamp  endTimestamp  dismissedUntil  snoozedUntil
```

Weekdays are seven lowercase named booleans, not a bitmask and not an array. All seven must be present on every write, because sending only the true ones leaves the rest at whatever they were, which is a merge, and this API does not merge.

THE DISCOVERY THAT UNBLOCKED TWO WEEKS OF FAILURE

> For a fortnight, every alarm OneAlarm created returned **200 and never appeared in the Eight Sleep app**. The cause was one line. New alarms are built by cloning an existing alarm rather than composing a payload, so that no field is ever guessed. The clone copied the template's `tags`, and the template carried:
>
> ```
> tags = ("temporary-mode", "oneOff-napMode")
> ```
>
> Eight Sleep's app does not list alarms carrying those tags under Alarms. The alarms existed, were enabled, and were ringing. They were simply invisible. Removing `tags` from the clone fixed it, confirmed on the bed on 17 August. A regression test fails if it is ever copied again.

The lesson that generalises: **a 200 is not a visible alarm**, and for a fortnight the search was in the payload shape because the create kept reporting success.

ROUTINES, WHICH IS HOW THEIR APP MODELS ALARMS

Eight Sleep's app renders alarms *inside* routines, and the routine object carries the days:

```
GET https://app-api.8slp.net/v2/users/{userId}/routines
PUT https://app-api.8slp.net/v2/users/{userId}/routines/{routineId}
    { days, bedtime, alarms, alarmsToCreate, enabled }
```

So a OneAlarm routine drives both: the alarm's time through the alarm endpoint, and the routine's days and switch through the routine endpoint. A routine with no alarm gets one added *inside* a routine whose days already match, through `alarmsToCreate`. Bedtime is read and handed back untouched: when someone goes to bed is not an alarm setting.

One caveat that cost time. **This account returns zero routines.** That code is committed, correct for an account that has them, and is not the mechanism for anything today. The invisible alarms were never a routine problem, they were the tags.

AN ALARM CREATED INSIDE A ROUTINE HAS NO DAYS OF ITS OWN

It takes them from the routine, so its `repeat.weekDays` comes back with every flag false. This has now caused two separate bugs: a create loop, because an empty day set matches no routine so the next sync creates another one; and a false "you will not be woken" warning, because a week-coverage check read an empty day set as covering no mornings. Both fixed. Worth knowing before writing anything that reasons about coverage.

GUARD RAILS ON THIS LEG

- Hard cap of **8 alarms** per account, so a runaway create loop cannot fill the bed.
- Never author `vibration`, `thermal`, level or pattern. Echo them.
- Never open a routine OneAlarm has no relationship with.
- Never write to an alarm OneAlarm does not own. Writing days to an unowned alarm is what once turned a real Monday-to-Friday schedule into every day.
- Never delete an alarm OneAlarm did not create.

THE OPEN PROBLEM

The one time change, and the current theory

The bug, confirmed on the bed on 17 August. An Eight Sleep alarm has one wall-clock time for its entire day set. There is no per-day time. So the first implementation of "tomorrow only at 09:40" wrote that time into the recurring alarm, and moved *every* weekday with it. The owner's report: *"instead of changing it for one time, it changes the entire Monday to Friday routine on Eight Sleep."* The only thing putting it back was a later sync, so if the app never ran again the whole week kept the wrong time.

What it does now, rebuilt on 18 August and not yet run on hardware. The override stops riding the routine's alarm and becomes an alarm of its own, using only mechanisms already confirmed working on this account:

1. The routine's own alarm is written with the routine's real time, always. Nothing about it changes.
2. A new alarm is created by cloning, covering one weekday, at the override time.
3. It is filed under a synthetic key shaped like a routine ID and carrying the date, `oneoff:<routine>:<yyyymmdd>`. While the morning is ahead, that key counts as a living routine and the alarm is protected. Once it passes, the key stops being generated, the existing orphan sweep finds the alarm, and deletes it because OneAlarm is recorded as having created it. Expiry needed no new machinery.
4. The routine's own alarm is skipped for that one morning through the native `skipNext` field, and only when the server's `nextTimestamp` confirms that morning is genuinely the next one.

> **A deliberate asymmetry worth arguing with.** Step 4 does not fall back to switching the weekly alarm off if the skip fails, although every other skip path in the codebase does exactly that and repairs itself on the next sync. Here it does not, because with no next sync the cost is a whole week with no alarm, against one morning ringing at the routine time instead of the override time. Between an alarm that rings early and an alarm that does not ring, this leg picks early and reports which happened.

The current theory about what their app actually does

Four raw dumps of the alarm object now show the same thirteen keys and no override field anywhere:

```
time  enabled  dismissedUntil  id  repeat  skipNext  skippedUntil
smart  snoozedUntil  snoozing  tags  thermal  vibration
( + nextTimestamp when the alarm is switched on )
```

But two alarms on this account are tagged `temporary-mode` and `oneOff-napMode`. One of those words is literally *oneOff*. And those tags cannot have originated with OneAlarm, because the clone copies a template's tags and the template is always an alarm the account already had. **So an alarm on this bed carried those tags before OneAlarm ever ran, which means Eight Sleep writes them.**

The hypothesis: their app implements `UPCOMING ALARM ONLY` as a **separate tagged alarm**, hidden from the alarm list and rendered instead as struck-through text on the card of the alarm it displaces. Which is, arrived at independently, the mechanism described above. If that is right, there is no field to find and the current design is the correct shape rather than a workaround.

It is testable without hunting for a field at all: set an override in their app, and see whether a new hidden alarm appears at that time.

> **A likely design error, found while writing this document, and the first thing an expert should check.** The API reference says a one-off Eight Sleep alarm is created by **omitting the `repeat` block entirely**. The implementation above instead creates a single-day *repeating* alarm, which is why it needs an explicit deletion the morning after. If a genuine one-shot works, it would need no cleanup, no synthetic key and no orphan sweep, and roughly a third of the one-off machinery could be deleted. The reason it was not used is mundane: the create path clones an existing alarm, and every clone carries a `repeat` block. This has not been tested.

SECOND SERVICE

Whoop, in brief

Auth is AWS Cognito reached through Whoop's own proxy, which injects the client ID, so the iOS app's client secret is not needed. MFA is supported. Access tokens last 24 hours.

```
GET https://api.prod.whoop.com/smart-alarm-bff/v1/schedule/all?apiVersion=7
PUT https://api.prod.whoop.com/smart-alarm-bff/v1/schedule/{id}?apiVersion=7
    { sleep_goal, day_of_week_list, time_zone_offset,
      enabled, latest_wake_time, alarm_mode }
```

Six keys, nothing echoed from the read, confirmed working against the live account. Two things about this leg are worth carrying:

- **The read and the write use different field names**, because `GET /schedule/all` is not the schedule. It is a rendered screen. Its top level carries keys like `delete_error_modal` and `schedule_button_component`. Five hours were spent concluding the write fields were fiction because they did not appear in that response.
- **An account holds several schedules.** This was assumed to be one for weeks, which forced a design where a single schedule's days were rewritten as the week turned over, and that is what once turned a real Monday-to-Friday schedule into every day.

Whoop's own one-off is mutually exclusive with the recurring schedule, per its own dialog, so OneAlarm refuses one time changes on this leg rather than silently destroying the week. It is reported as a refusal with a reason.

THE CENTRE OF THIS DOCUMENT

Soll-Ist-Analyse

Every main function, what it is supposed to do, and what it actually does as of 18 August 2026. Assessments and proposed fixes are in the numbered notes below the table.

Read the status column carefully. `WORKS` means confirmed on the real hardware, by the owner, on a stated date. `UNVERIFIED` means the code exists and passes tests against a stub server and *has never run against a real device*. `BROKEN` means known not to work.

FUNCTION	SOLL	IST, 18 AUGUST	STATUS	NOTE
iPhone alarm rings	Rings at the master time through Silent and Focus, phone locked	Confirmed on device, 16 August, locked and face down	WORKS	
Eight Sleep: routine time	Change a routine time, the bed follows	Confirmed on the bed, 17 August 14:21. Owner: *"Ok eightsleep is save"*	WORKS	
Eight Sleep: create a missing alarm	A routine with no alarm on the bed gets one	Confirmed. `EVERY WEEKEND 09:55` created by the app and visible in theirs	WORKS	1
Eight Sleep: detect a foreign override	Tell the user when the bed is firing at a time its schedule does not describe	Confirmed 18 August from a live screenshot. The panel named the overridden alarm by itself	WORKS	2
Whoop: schedule time	Change a routine, the strap follows	Confirmed, new time visible in the Whoop app	WORKS	
Whoop: one schedule per routine	Two routines drive two separate schedules	Confirmed 17 August after the single-schedule assumption was refuted	WORKS	
Eight Sleep: one time change	Tomorrow only moves. The rest of the week is untouched	**Was broken:** moved the entire Monday-to-Friday series. Rebuilt 18 August, then corrected the same night to use Eight Sleep's own one-shot, a POST with no `repeat`, confirmed in two sources. **Never run on hardware**	UNVERIFIED	3
Eight Sleep: override self-cleanup	The extra alarm is deleted on the first sync after its morning	Built. Reuses the existing orphan sweep. Never run	UNVERIFIED	4
Eight Sleep: skip one morning	Native `skipNext`, weekly alarm stays switched on	Built and checked against `nextTimestamp` moving. Never run on hardware	UNVERIFIED	5

FUNCTION	SOLL	IST, 18 AUGUST	STATUS	NOTE
iPhone: one time change	Tomorrow moves, every other morning stays armed	**Was wrong:** stood every routine alarm down until the override passed, so bending next Saturday left Monday to Friday with no phone alarm. Rebuilt 18 August. Never run	UNVERIFIED	6
Whoop: create a schedule	A routine with no schedule gets one	A ladder of three candidate requests, best-evidenced first. No public capture of a create exists anywhere	UNVERIFIED	7
Resilience to change	Editing a routine's days keeps the alarm it already owns	**Corrected 18 Aug.** Merging and narrowing keep the owned alarm: the recorded link is consulted before day sets, so the earlier "orphans both alarms" note was stale. The gap is the **first** adoption, which still needs exact day-set equality	PARTLY	8
No invisible alarms	Nothing rings that the owner cannot see and switch off	**Two remain** on the account from before the tags fix, at `05:55` weekdays and `09:56` Sa/Su, invisible in the Eight Sleep app. One is now off	BROKEN	9
Whoop: one time change	Tomorrow only, as on the other two legs	**Refused by design**, with the reason reported. Whoop's own one-off is mutually exclusive with the recurring schedule	BY DESIGN	10

NOTES

Assessment and proposed fix, per row

1

Create works, but only because it copies rather than composes

ASSESSMENT

New alarms are clones of an alarm the account already has, with two fields changed. Nothing is composed from a spec, which is what makes it safe: the reference documentation for this API contradicts itself about field names thirty lines apart, `vibration.powerLevel` against `vibration.level`, and that contradiction would end up in the bed's temperature.

CONSEQUENCE

An account with zero alarms cannot be bootstrapped. The app asks the owner to create one by hand, once, and says so plainly.

2

Detection works without knowing where the override lives

ASSESSMENT

This was solved by computing rather than searching. `time` is the weekly wall clock and `nextTimestamp` is the absolute instant the alarm next fires. An override necessarily makes those two disagree, *whatever field carries it and wherever it lives*. Three separate attempts to have the owner find the field by reading raw dumps failed, because reading cannot distinguish "no override was set" from "it is not on this object".

WORTH GENERALISING

When a response keeps coming back the same, the missing thing is not another dump. It is a derived value that is impossible unless the state you are hunting for exists.

3 The one time change: most likely cause of the next failure

ASSESSMENT

The rebuild is sound in shape and completely untested against the real API. Three specific things could break it. First, the create could be refused for a single-day payload, in which case the routine is left untouched and the row says so. Second, an alarm created standalone might not appear in their app, which is exactly the failure mode of the tags bug, although a standalone create was confirmed visible on 17 August. Third, the native skip could silently do nothing, in which case both alarms ring and the earlier one wins.

PROPOSED FIX

Before anything else, test whether omitting the `repeat` block produces a genuine one-shot alarm, as the API reference claims. If it does, delete the single-day-repeating approach along with its synthetic key and its cleanup, and use the native one-shot. That is a materially simpler design and it removes note 4 entirely.

4 Self-cleanup is the highest-consequence unverified path

ASSESSMENT

If the created override alarm is not deleted after its morning, it becomes a weekly alarm at the wrong time, which is the exact failure the whole rebuild exists to prevent. It depends on the DELETE endpoint, which is a convention rather than a capture: the PUT path with a different verb. If DELETE is refused, the app falls back to switching the alarm off, which is safe but leaves visible litter.

PROPOSED FIX

Confirm the DELETE with one live call. If it is refused, the one-shot approach in note 3 becomes not just simpler but necessary, because a one-shot needs no delete at all.

5 skipNext is a field name we have never exercised

ASSESSMENT

`skipNext` and `skippedUntil` appear in every dump, always `0` and epoch. Their behaviour is known from their names and nothing else, which normally would not be enough to act on. What makes it acceptable is that the guess is falsifiable: if the field does what its name says, `nextTimestamp` moves past the skipped morning. If it does not move, nothing was skipped and the old behaviour runs instead. One wasted request, no damage.

NOTE FOR THE EXPERT

The live screenshot on 18 August showed an override in force with `skipNext = 0`, which rules `skipNext` out as the mechanism behind `UPCOMING ALARM ONLY`.

6 The phone leg was wrong for a documented reason, which is its own lesson

ASSESSMENT

AlarmKit offers `.never` and `.weekly` and nothing in between, so "this Saturday and not every Saturday" cannot be expressed as one alarm. The original answer was to arm a single alarm for the override morning and stand every routine down. Because the flag meant "this routine has an override somewhere ahead" rather than "tomorrow", bending next Saturday from a Monday left the whole working week with no phone alarm at all. On the one leg that exists precisely because it needs no account and no network.

WHAT MADE IT SURVIVE

It carried a long comment explaining the trade-off and a test pinning the behaviour. A documented defect reads, to the next person, as a decision already weighed. It is expressible as *two* weekly alarms, which is what it does now.

RECOMMENDATION

When you write a defect down instead of fixing it, write down what would unblock it in the same breath, and re-check that list whenever anything new lands.

7 The Whoop create is the least evidenced thing in the app

ASSESSMENT

It exists because the owner deleted every schedule on his account and then could not recreate one from Whoop's own app. No public source documents a create or a delete. The most thorough public capture of the app deliberately exercised smart alarm CRUD and recorded only an update, which is weak evidence that the others are not plain REST.

DESIGN

A ladder of three candidates, most evidenced first: `PUT /schedule/{a-fresh-uuid}`, on the theory that the update endpoint is an upsert, since client-minted IDs are the standard shape for that. Every rung reports its status. A create cannot destroy anything and the account is capped at 7 schedules.

EXPLICIT BAN

No fourth rung and never a DELETE without a capture behind it. A summarising web fetch once invented a clean create and a matching delete, neither of which existed in the file it claimed to be reading. It was caught by downloading the raw source.

8

Identity by content similarity: the real architectural problem

ASSESSMENT

This is the most important row in the table and the owner diagnosed it himself. A routine finds its alarm three ways, in order: a recorded link, then **exact day-set equality** for an alarm nobody owns yet, then nothing. Exact equality is a good rule for a first meeting and a poor one for a lifetime. It works perfectly from a matched starting state, which is why the app looks reliable when set up carefully and fragile the moment anything changes.

CORRECTED 18 AUGUST

This note previously said merging orphans both alarms and narrowing strands one. Tested rather than assumed, and both are stale: once a routine owns an alarm through the recorded link, the link is consulted before any day set, so merging keeps the survivor's alarm and rewrites its days, narrowing reshapes it, and the merged-away alarm becomes an orphan that provenance resolves. The notes described the world before ownership was recorded and were never revisited.

WHAT IS GENUINELY OPEN

The **first** adoption, not the lifetime. A routine with no link finds its alarm only by exact day-set equality, so a routine created after its alarm has drifted never adopts it, and an alarm nobody has adopted stays unowned forever. That is the owner's own diagnosis stated precisely, and it is a much smaller problem than the one this row used to describe.

PROPOSED FIX

Content similarity is a fine way to *propose* a pairing and a bad way to *maintain* one. Mature sync systems use it once, at adoption, with an explicit user confirmation, and never again as identity. This app has the ID mapping already; what it lacks is a confirmation step at adoption and a diff shown before the write. A "week coverage diff" that says what will change before it changes is the concrete next piece of work, and it is more valuable than any further one-off work.

9

Two invisible alarms are still on the account

ASSESSMENT

Created by OneAlarm before the tags fix, so they carry the nap tags and cannot be seen or switched off in the Eight Sleep app. They predate the provenance record, so the app cannot prove it made them and therefore will not delete them, correctly. The app offers to switch them off and is waiting on the owner's approval, because it writes to his bed.

PROPOSED FIX

Approve the switch-off. They are the residue of a fixed bug, not a live one, but an alarm that rings and cannot be seen is the worst thing in this system and should not be left in place.

10

Whoop one-offs are refused, and refusing is the right answer today

ASSESSMENT

Whoop's own dialog states that its one-off and its recurring schedule are mutually exclusive. Implementing a one-off by editing the recurring schedule would destroy the week, which is precisely the Eight Sleep bug in a service with less room to recover. A separate single-day schedule would need a create, which is unverified, and a delete, which is banned without a capture.

PROPOSED FIX

Leave it refused until the create and delete are captured. A refusal that is reported is a feature. A silent destruction of a week is not.

THE GRAVEYARD

What was tried that did not work

Kept in full, because the failures are more instructive than the fixes and because a new engineer will otherwise re-run them.

On the services

- **Writing the override into the recurring alarm's `time`.** Moved every morning that routine covered. The founding bug of the current work.
- **Trusting a single source for a payload shape.** Three times. Whoop's field names, Whoop's read shape, and Eight Sleep's routine payload, where `dayOffset` was read as the number `0` from one capture and is a string enum `"Zero"` in both implementations that spell it. Two sources, or it is not a capture.
- **Inferring absence from the wrong object.** Whoop's field names were declared fiction because a GET did not contain them. That GET returns a rendered screen. Eight Sleep was declared not to name its beds because a parser did not read a name, when nobody had looked at the object. Both existed. Cost: five hours and a wrong entry in the notes.
- **Trusting a summarising fetch instead of raw source.** A clean `POST` create and matching `DELETE` for Whoop were invented by a summarisation, neither present in the file it claimed to be reading.
- **Assuming Whoop holds one schedule per account.** It holds several. The single-schedule assumption forced a design that rewrote one schedule's days as the week turned over, which turned a real Monday-to-Friday schedule into every day.
- **Assuming routines were hiding the invisible alarms.** This account has no routines at all. It was the tags.

On method

- **Changing several things at once, then drawing a conclusion.** Four changes in one commit, and when the next round also failed, "it is not the body" was concluded from two attempts that were the same shape twice.
- **A test that mutates a live account.** Turning on all seven days to test the ring rewrote a real Whoop schedule from Monday-to-Friday into every day, silently, because the write replaces.
- **A retry that silently changed a setting the user chose.** A Whoop write fell back to a different `alarm_mode` to get past a suspected enum error, which would have overwritten a mode set by hand ten minutes earlier.
- **A safety screen that overstated itself.** The write preview claimed to be built by the same code as the real request. It was a reconstruction, and it omitted the field

most likely to be causing the refusal. A gate that lies is worse than no gate, because it is where you go to rule something out.

- **A green test asserting the broken behaviour.** The one-off had a test named for what the code did rather than for what the feature is. It passed, and what it asserted was the bug. It is the reason nobody looked there.
- **Diagnosing from the wrong layer.** CI running zero jobs was diagnosed as a billing problem, and the owner was walked through checking spending limits and making a repository public. The cause was invalid workflow YAML.

The pattern that dominates recent work

> On 18 August the one-off was rebuilt correctly and then **eight separate defects were found between the working code and the user's screen**, and not one was in the reasoning about alarms:
>
> - The status line discarded its detail message whenever a sync went completely fine, so a one time change that worked perfectly printed nothing about it.
> - The write preview read the wrong time field and showed a time the app does not send.
> - The build stamp lived three taps into a settings screen while the question it answers is asked on the home screen.
> - The stray-alarm check did not know about the new kind of ownership, so it told the user to delete the alarm the feature had just created.
> - Verification read back the alarm it had just skipped, so a perfect write reported a mismatch.
> - The confirmation screen covered the row and said "nothing left to do" over the answer.
> - A week-coverage check read an alarm with no day flags as covering no mornings, producing a false "you will not be woken" on a working setup.
> - Both new checks vanished silently when a read failed, and silence is indistinguishable from "nothing to report".
>
> **The transferable finding: the logic has consistently been right and the path from it to a person is where everything breaks.** Work is not finished at the return statement, and not at the receipt either. It is finished at the pixel the user is looking at when they stop.

IF YOU ARE PICKING THIS UP

Suggested order of work

1. **Test the native one-shot**, note 3. One request. If it works, roughly a third of the one-off machinery can be deleted and note 4 disappears with it.

2. **Confirm or refute the tagged-alarm theory** for `UPCOMING ALARM ONLY`. Set an override in Eight Sleep's app and look at whether a new hidden alarm appears. One minute, no field hunting.
3. **Answer** `skipNext`, note 5. It is already instrumented; it needs one run.
4. **Then the identity model**, note 8. This is the real architectural work and everything else is comparatively cosmetic.
5. **Whoop create**, note 7, last. It only matters when an account has no schedule, which is rare and recoverable by hand.

> **House rules that are not negotiable, and each one was paid for.** Reverse-engineered surfaces are personal-use and single-account only. Never send destructive, firmware or wipe commands. Respect rate limits, single-digit requests per second. All credentials live in the iOS Keychain, never in `UserDefaults`, never logged, never committed. No telemetry and no backend. Every device write goes through a preview gate in debug builds. When stuck, print the response rather than reasoning about it: both Whoop breakthroughs came from that, and six rounds of reasoning beforehand produced six wrong answers.